Báo cáo thực hành DevOps

Deploy ứng dụng, tối ưu pipeline
và xử lý sự cố sau release

Nguyễn Tân Tiến
Ứng tuyển vị trí DevOps Engineer

Bài làm gồm Task 0 bắt buộc và hai nội dung tự chọn là Task 4, Task 5. Task 0 dùng game 2048 nguồn mở để tập trung vào đóng gói image, truy vết phiên bản và deploy. Task 4 và Task 5 là phương án xử lý tình huống theo đề, không mô tả một lần vận hành thực tế.

Tôi chọn Jenkins để build và test image; Argo CD đọc cấu hình từ Git để deploy lên Kubernetes. Khi release hoặc rollback, phiên bản được cập nhật trong Git; Jenkins không cần kubeconfig.

Nội dung và vị trí tra cứu

Phần việc	Trang
Task 0 · Kiến trúc, cấu hình và rollback	2–3
Task 0 · Cách chạy, xử lý lỗi và kết quả test	4–5
Task 4 · Phân tích pipeline 25 phút và bốn đề xuất	6–7
Task 5 · Điều tra, khôi phục dịch vụ và tổng kết sự cố	8–10

Mã nguồn và phạm vi xác minh

github.com/Pay2Winn/concung-devops-case-study
`README.md` mô tả bài nộp; `lab/README.md` hướng dẫn tái dựng môi trường.

Tại local, tôi đã chạy Jenkins build/test với `DEPLOY=false`, deploy Helm thủ công và kiểm tra Argo CD sync riêng. Log và ảnh nằm trong `lab/evidence/`. Chưa chạy toàn bộ pipeline deploy hoặc thử rollback qua Git.

Luồng CI/CD với Jenkins và Argo CD

Dùng game 2048 của Gabriele Cirulli, giấy phép MIT, không phải game tự phát triển. Nguồn được pin tại commit `478b6ec346e3787f589e4af751378d06ded4cbbc`. BuildKit lấy nguồn; Dockerfile chỉ copy file tĩnh, không chạy script upstream. Giữ giấy phép tại `LICENSE.txt`.

Base image Nginx được pin bằng digest. Container chạy với UID/GID `10001:10001`, không có quyền root. Runtime chỉ cho ghi vào `/tmp`, phần còn lại chỉ đọc. Ứng dụng tĩnh không cần thêm backend.

Các stage trong Jenkinsfile

1. **Checkout và build.** Lấy commit của repo bài làm, build một lần. Tag dùng 12 ký tự đầu commit SHA; full commit được ghi vào image label và trả tại `/version`. Commit bài làm khác commit nguồn game.
2. **Test và lint.** Chạy `tests/smoke.sh` trên image vừa build, rồi lint và render Helm chart. Stage sau chỉ chạy khi stage trước pass.
3. **Kiểm tra trước deploy.** Khi bật `DEPLOY=true`, nạp cấu hình từ Jenkins Folder và ghi nhận commit hiện tại của `main`. Commit đó phải trùng commit đã build.
4. **Push và cập nhật Git.** Push image, lấy digest thuộc đúng repository và test lại image pull từ registry. Ghi repository/tag/digest vào `environments/dev/values.json`, commit rồi push. Dừng nếu remote thay đổi; không force push hoặc rebase để bỏ qua conflict.
5. **Xác minh Argo CD và phiên bản.** Application phải `Synced/Healthy` tại đúng Git commit vừa push; game và full commit tại `/version` phải khớp bản dự kiến. Đọc lại trạng thái sau kiểm tra; Healthy của bản cũ không đủ.

Jenkins cập nhật Git, Argo CD deploy lên Kubernetes

Jenkins build/test, push image vào registry rồi cập nhật repository/tag/digest trong Git. Argo CD đọc values từ Git, render Helm chart và apply tài nguyên lên Kubernetes. Jenkins không giữ kubeconfig hay chạy `kubectl apply`; pipeline gọi API Argo CD để xác nhận đúng commit đã deploy.

`DEPLOY=false` chỉ build/test. Muốn deploy phải bật `DEPLOY=true` trên trusted `main`, được phép push và cấu hình đủ Credentials, Argo CD, HTTPS. PR không được deploy. Commit values không tự bật deploy lần sau. Token Argo CD chỉ có quyền đọc; pipeline không bỏ qua xác minh TLS.

Cấu hình, scale và rollback

Helm chart chứa templates cho Deployment, Service và ConfigMap; values chứa phiên bản image và cấu hình môi trường. Argo CD render chart rồi apply lên cluster. Không chạy thêm Helm release để cùng quản lý những tài nguyên này.

Thiết lập của lab	Ý nghĩa và giới hạn
Replicas	`replicaCount: 2` trên kind một node. Thay Pod lần lượt khi rolling update; không bảo vệ ứng dụng khi node lỗi.
Rolling update	`maxUnavailable: 0`, `maxSurge: 1`; node cần đủ tài nguyên cho Pod thứ ba. Pod mới phải Ready trước khi bỏ Pod cũ.
Cấu hình môi trường	ConfigMap cấp `APP_ENV=dev`. Checksum cấu hình đổi sẽ kích hoạt rollout. `/version` trả môi trường và commit.
Health checks	Startup probe kiểm tra khởi động; readiness quyết định Pod có nhận traffic; liveness phát hiện container cần restart. Cả ba gọi `/health`.
Resource requests/limits	Mỗi container có requests CPU `50m`, RAM `32Mi`; limits `250m`, `128Mi`. Requests dùng khi schedule Pod; limits giới hạn tài nguyên container.
Container security	Không cho privilege escalation, drop toàn bộ capabilities, dùng seccomp RuntimeDefault. Không mount Kubernetes API token. `/tmp` dùng RAM, tối đa 64 MiB.
Truy cập	NodePort `30080` được kind ánh xạ ra `localhost:8083`. Lab không cần Ingress hoặc mở dịch vụ ra Internet.

Scale bằng values trong Git

Đổi `replicaCount` trong values rồi commit vào Git. Trước khi tăng, kiểm tra tải và tài nguyên còn trống trên node. Chỉ thêm autoscaling khi có metrics, Metrics Server và mục tiêu cụ thể. Thêm Pod trên một node không tạo multi-node HA.

Rollback theo công cụ đang quản lý

Với Argo CD, khôi phục repository/tag/digest và config ổn định trong Git. Xác nhận Application `Synced/Healthy` tại commit rollback, rồi kiểm tra `/version` và chức năng. Nếu chỉ dùng Helm thủ công, xem `helm history`, chọn revision và chạy `helm rollback` với `--wait`. Không dùng hai cách cùng lúc. Giữ image cũ trong registry để còn rollback.

Chạy và kiểm tra tại máy lab

Cần Docker Desktop ở chế độ Linux containers, BuildKit, Bash, Git, Helm, kind và kubectl; dự trù khoảng 8 GB bộ nhớ cho cả lab. Các lệnh dưới chạy từ thư mục gốc repo. Việc tải nguồn và image cần Internet khi tái dựng.

```
# Chạy riêng ứng dụng, chưa dựng Kubernetes
 docker compose up -d --build app
 curl --fail http://localhost:8082/health
 curl --fail http://localhost:8082/version

# Build và kiểm tra image thật
 docker build --platform=linux/amd64 \
   --build-arg REVISION=local -t concung-2048:local .
 MSYS_NO_PATHCONV=1 sh tests/smoke.sh \
   concung-2048:local local

# Kiểm tra cấu hình
 helm lint helm/demo -f environments/dev/values.json
 helm template demo helm/demo \
   -f environments/dev/values.json
 docker compose --profile agent config --quiet
```

`MSYS_NO_PATHCONV` ngăn Git Bash trên Windows đổi đường dẫn Linux truyền cho Docker. Smoke test kiểm tra đóng gói và HTTP, không kiểm thử toàn bộ luật chơi.

Nối các thành phần theo lab/README.md

1. **kind và registry.** Kiểm tra tài nguyên trùng tên trước khi tạo; không xóa để ép chạy. Dùng kubeconfig riêng tại `.runtime/`, không dùng context công ty. Chuẩn bị image có tag/digest thật; tag toàn số 0 ban đầu chỉ là placeholder.
2. **Jenkins.** Controller có 0 executor; agent `docker-agent` có một executor. Tạo Folder và Multibranch Pipeline, Git Tool `Default`. Cấu hình thường dùng Folder Properties; token và chứng chỉ dùng Credentials.
3. **Argo CD.** Chuẩn bị HTTPS với CA tin cậy, token có hạn chỉ đọc Application và values trỏ đúng image tồn tại. Chạy `DEPLOY=false` trước, chỉ bật `true` khi đã được phép push Git.

Dịch vụ trên host	Địa chỉ local
Jenkins · Registry	`http://localhost:8081` · `localhost:5001`
Game Compose · Kubernetes	`http://localhost:8082` · `http://localhost:8083`
Argo CD	`https://localhost:8443`

Kết quả deploy và kiểm tra

Sau khi deploy, tôi đối chiếu commit, digest và chơi thử game qua cổng Kubernetes để xác nhận ứng dụng chạy đúng bản.

Hiện tượng	Kiểm tra trước khi thay đổi
Build lỗi	Log bước lỗi, commit nguồn cố định, mạng tới GitHub/registry và dung lượng Docker host.
Agent offline	Tên, label, secret, WebSocket và log agent. Mạng `kind` phải tồn tại trước khi bật profile agent.
`ImagePullBackOff`	Image/tag/digest có tồn tại, registry và mirror của kind; xem sự kiện Pod.
`Pending` hoặc restart	Requests, chỗ cho Pod mới, log hiện tại/lần trước, probes, bộ nhớ và quyền ghi `/tmp`.
Git, TLS, 401/403	Remote, branch protection, hostname chứng chỉ, CA, hạn và quyền token. Không force push hoặc tắt xác thực TLS.
Sai phiên bản	So full commit tại `/version`, digest và commit Argo đồng bộ. Phân biệt cổng 8082 với 8083.

Đã deploy thành công, có bằng chứng

Jenkins build #2 thành công với `DEPLOY=false`: build image, smoke test và Helm lint/render đều pass. Sau đó, tôi push image vào registry local và deploy bằng Helm. Hai Pod Ready; digest và `/version` khớp source commit. Game chơi được bằng phím; màn hình mobile không tràn ngang.

Argo CD tiếp quản ba tài nguyên và đạt `Synced/Healthy` tại commit `b4996d303104`. Tôi ghi đè ba tham số image trong Application local, không sửa values trong Git. Agent dùng token chỉ đọc, xác minh TLS và kiểm tra trạng thái trước/sau khi gọi ứng dụng. Bằng chứng: `lab/evidence/`.

Tìm nguyên nhân pipeline mất 25 phút

Theo đề bài, pipeline mất khoảng 25 phút, Docker build thường lỗi và developer phản ánh deploy chậm. Đây là tình huống cần phân tích, không phải benchmark của demo 2048. Cần log từng stage trước khi kết luận Docker build là bước chậm nhất.

Dùng Jenkins timestamps để ghi thời gian từng stage và queue time. So nhiều build cùng commit hoặc thay đổi tương đương; tách cache hit và cache miss. Tính cả build lỗi và retry để thấy thời gian thực tế từ lúc bắt đầu đến khi deploy thành công.

Khoảng cần tách	Bằng chứng và câu hỏi điều tra
Queue time	Agent online, số executor và tài nguyên còn trống. Job thiếu agent hay phải chạy lần lượt vì cùng deploy một môi trường?
Checkout, dependencies	Dung lượng repo, tốc độ tải, timeout và authentication. Dependencies không đổi có bị tải lại ở mỗi build không?
Docker build	Thời gian từng layer, cache hit/miss, build context và lỗi cụ thể. Đổi source code có làm mất cache của layer dependencies không?
Test và push	Thời gian từng nhóm test, kích thước image và phản hồi registry. Chậm ở test hay truyền image?
Argo CD sync và Pod Ready	Tách thời gian Argo CD phát hiện commit mới, sync đúng revision, pull image, schedule Pod và startup/readiness probes. Xác định chậm ở sync hay ứng dụng khởi động.

Phân loại lỗi build trước khi retry

Mã hoặc cấu hình: sửa Dockerfile, file thiếu hoặc thư viện không tương thích. Retry không sửa được nguyên nhân. Giữ log lỗi đầu tiên.

Tài nguyên máy build: kiểm tra bộ nhớ, đĩa và build tranh CPU. Chỉ tăng năng lực khi có bằng chứng; thêm executor trên cùng máy có thể chậm hơn.

Lỗi bên ngoài: mạng, DNS hoặc registry có thể lỗi tạm thời. Retry có giới hạn; không che lỗi quyền, chứng chỉ hoặc mã.

Chọn nút thắt để thử trước

Tôi sẽ thử tối ưu bước đang làm cả pipeline phải chờ lâu nhất. Mỗi lần chỉ đổi một yếu tố, giữ các kiểm thử bắt buộc rồi so thời gian và tỷ lệ lỗi.

Bốn thay đổi cần thử và đo

Các đề xuất dưới đây phụ thuộc nút thắt ở trang 6. Không cần áp dụng cả bốn nếu phần thời gian tương ứng không đáng kể.

1. Cache dependencies và Docker layers

Với ứng dụng có dependencies, copy lockfile và cài dependencies trước, copy source code sau. Dùng `.dockerignore` loại file không cần khỏi build context. Docker daemon local giữ được cache; agent mới cần cơ chế cache riêng nếu muốn tái sử dụng.

Dùng khi mỗi build tải/cài lại dependencies không đổi hoặc gửi context lớn. **Đo** thời gian từng layer, kích thước context và cache hit rate; so cả cache hit/miss. Demo 2048 chỉ copy static assets, không cài dependencies.

2. Chạy song song các stage độc lập

Helm lint/render có thể chạy cùng Docker build nếu agent đủ tài nguyên. Container test phải sau build; push phải sau test; cập nhật Git phải có digest. Bước xác minh chỉ pass khi Argo CD sync đúng revision và smoke test đạt. Các build cùng deploy một môi trường vẫn phải chạy lần lượt.

Dùng khi có stage độc lập và đủ CPU/RAM. **Đo** tổng thời gian, queue time và tỷ lệ lỗi do tranh tài nguyên. Jenkinsfile hiện chạy tuần tự, không cho build đồng thời; parallel stages là đề xuất.

3. Giảm kích thước runtime image

Dùng multi-stage build để chỉ copy build artifacts cần chạy vào image cuối. Demo đã tách stage lấy source, chỉ copy static assets vào Nginx; không chạy script upstream hay đưa build tools vào runtime. Không thêm compile step cho ứng dụng không cần compile.

Dùng khi image chứa build tools hoặc file thừa. **Đo** kích thước image và thời gian build/push/pull. Giữ base image được pin bằng digest và các thiết lập container security.

4. Build một lần, deploy cùng digest

Build từ commit đã chọn, test image đó, push rồi deploy bằng digest. Khi promote qua staging và production, đổi runtime config, không build lại. Pipeline hiện ghi full commit vào image và test lại image pull từ registry trước khi cập nhật Git.

Dùng khi pipeline build lại cùng phiên bản cho từng môi trường. **Đo** số lần build/release, tổng thời gian và digest giữa các môi trường. Public image có sẵn giúp kiểm tra pull/deploy, không thay test image chứa source code mới.

Website chậm sau release

Website chậm và CPU tăng sau deploy cần được kiểm tra ngay. Tôi sẽ so logs và metrics trước/sau release, không kết luận bản mới gây lỗi chỉ vì sự cố xuất hiện cùng thời điểm.

Xác nhận ảnh hưởng và mở incident

Đối chiếu alert với phản ánh người dùng. Ghi thời điểm kèm múi giờ, chức năng bị ảnh hưởng, phạm vi và phiên bản. Gọi on-call, báo team/lead; tạm dừng rollout nghi lỗi và các release không liên quan để tránh thêm thay đổi trong lúc điều tra.

Chọn SEV theo mức ảnh hưởng và quy định của tổ chức. Ví dụ đề xuất: SEV1 khi chức năng chính không dùng được trên diện rộng; SEV2 khi dịch vụ suy giảm đáng kể hoặc ảnh hưởng cục bộ. Không xác định SEV chỉ từ CPU cao; nâng mức nếu ảnh hưởng tăng.

Phân vai và ghi timeline

Chọn Incident Commander để điều phối, người xử lý kỹ thuật và người cập nhật thông tin. Team nhỏ có thể kiêm nhiệm nhưng phải rõ ai quyết định, ai thao tác và ai nhận bàn giao. Không để nhiều người cùng sửa một thành phần mà không trao đổi.

Đối chiếu metrics, logs và traces

- **Metrics:** so trước/sau release ở traffic tương đương: HTTP 5xx, latency p95, CPU, RAM, restart count và CPU throttling. p95 là ngưỡng latency mà 95% request không vượt quá.
- **Logs:** đối chiếu timestamp, request ID và phiên bản; lấy previous container logs nếu có restart. Kiểm tra thay đổi source code, config và dependencies.
- **Distributed traces, nếu có:** tìm bước chậm ở ứng dụng, database hay dịch vụ ngoài. Kiểm tra query, connection pool và retry; không giả định hệ thống đã có tracing.

Phân biệt CPU cao và OOMKilled

`OOMKilled` cho biết container bị dừng do thiếu RAM, không phải do CPU cao. Kiểm tra termination reason, Pod events và RAM so với limit. Exit code `137` riêng lẻ chỉ cho biết tiến trình bị SIGKILL, chưa đủ kết luận OOM. Với CPU cao, xem traffic, throttling và bước xử lý chậm trước khi scale.

Chỉ điều tra trong môi trường được cấp quyền. Che dữ liệu nhạy cảm; giữ bằng chứng cần thiết trước khi khởi động lại hoặc thay thế container.

Rollback hay hotfix?

Ưu tiên khôi phục dịch vụ, không đợi tìm đủ nguyên nhân mới xử lý. So thời gian và rủi ro của từng phương án; tuân thủ quyền thay đổi khẩn cấp của tổ chức. Ghi người thao tác, người phê duyệt và thời điểm đánh giá lại.

Rollback khi bản ổn định còn dùng được

Tôi chọn rollback khi có bằng chứng lỗi liên quan đến release mới và rollback an toàn, nhanh hơn hotfix. Trước đó, xác nhận image/config cũ còn sẵn và tương thích với dữ liệu, database schema, dependencies hiện tại. Rollback ứng dụng không tự rollback dữ liệu; schema không tương thích có thể khiến bản cũ không chạy.

Với Argo CD, khôi phục digest/config ổn định trong Git, review khẩn cấp rồi xác nhận `Synced/Healthy` tại commit rollback. Nếu chỉ dùng Helm thì rollback revision phù hợp. Không để hai công cụ cùng quản lý. Argo CD có thể ghi đè thay đổi thủ công theo Git; nếu cần thao tác khẩn cấp, phải kiểm soát sync và cập nhật lại Git ngay sau đó.

Nếu hệ thống có sẵn feature flag hoặc traffic routing, có thể tắt tính năng lỗi hay chuyển traffic sang bản tốt. Không rollback về bản không tương thích dữ liệu.

Hotfix hoặc giảm ảnh hưởng khi ít rủi ro hơn

Chọn hotfix nếu nguyên nhân đủ rõ, thay đổi nhỏ, test được và ít rủi ro hơn rollback. Deploy trên phạm vi nhỏ nếu hệ thống hỗ trợ; xác định khi nào phải dừng và rollback trước khi mở rộng.

Tăng tài nguyên hoặc restart chỉ xử lý tạm thời nếu ứng dụng vẫn memory leak. Trước khi scale, xem tài nguyên còn trống, traffic, CPU limits và tải lên dependencies. Ghi thời điểm/kết quả từng thay đổi; không sửa nhiều thứ cùng lúc rồi đoán tác dụng.

Kiểm tra trước khi xác nhận dịch vụ phục hồi

1. **Đúng phiên bản.** Ghi commit Git, digest và cấu hình; đối chiếu phiên bản đang phục vụ thay vì chỉ nhìn pipeline xanh.
2. **Đúng chức năng.** Chạy lại luồng nghiệp vụ bị ảnh hưởng. Pod Ready và HTTP 200 chưa chứng minh luồng đó hoạt động đúng.
3. **Latency và error rate.** Trở về SLO hoặc baseline đã thống nhất nếu chưa có SLO.
4. **Ổn định dưới traffic đại diện.** Theo dõi CPU, RAM, restart và backlog. Chưa đạt thì tiếp tục xử lý, đổi phương án hoặc gọi chuyên gia.

Cập nhật và theo dõi sau sự cố

Nhóm chọn một người cập nhật để tránh báo tin khác nhau. Tôi đề xuất SEV1 cập nhật mỗi 15 phút, SEV2 mỗi 30 phút và khi có thay đổi lớn; chưa có tiến triển vẫn báo. Tần suất này cần thống nhất trong nhóm, không phải quy định chung.

Thời điểm	Nội dung cần có
Kích hoạt	Mã sự cố, thời điểm, mức nghiêm trọng, phạm vi ảnh hưởng, người chỉ huy và giờ cập nhật kế tiếp.
Đang xử lý	Ảnh hưởng hiện tại, bằng chứng, hành động và kết quả. Chưa rõ nguyên nhân hoặc giờ phục hồi thì nói rõ, không hứa thiếu căn cứ.
Theo dõi	Biện pháp đã áp dụng, phiên bản và kiểm tra chức năng. Nêu dịch vụ đang được theo dõi, chưa tuyên bố nguyên nhân đã sửa dứt điểm.
Kết thúc ứng cứu	Người chỉ huy xác nhận phục hồi; ghi thời gian ảnh hưởng, rủi ro còn lại, người theo dõi và lịch tổng kết.

Nội bộ dùng kênh sự cố đã thống nhất, chẳng hạn Teams. Khách hàng nhận thông tin qua người được ủy quyền. Không đưa secret, thông tin nhận dạng hoặc log nhạy cảm vào thông báo.

Postmortem sau khi dịch vụ ổn định

Tôi cùng team xem lại timeline, ảnh hưởng đến người dùng và những bước xử lý còn chậm. Mục đích là cải thiện hệ thống và cách phối hợp, không quy trách nhiệm cá nhân. Rollback thành công chưa có nghĩa đã tìm ra root cause; vẫn cần đối chiếu logs, traces hoặc tái hiện lỗi an toàn ở môi trường test.

Theo dõi action items

Team thống nhất việc sửa root cause, bổ sung regression test/load test, điều chỉnh alerts và cập nhật runbook. Nếu phù hợp, thêm canary deployment và diễn tập rollback. Mỗi việc có owner, priority, due date và cách kiểm tra kết quả. Team theo dõi tiến độ, xử lý việc quá hạn và xác minh lỗi không tái diễn sau khi sửa.

Nếu chưa rõ nguyên nhân, tôi bàn giao bằng chứng và thống nhất người tiếp tục điều tra. Nhóm theo dõi thời gian phát hiện, xác nhận, khôi phục dịch vụ và kiểm tra lỗi có tái diễn hay không.

Tài liệu đối chiếu Task 0 và Task 4

- Repo bài làm: github.com/Pay2Winn/concung-devops-case-study
- 2048 upstream và giấy phép MIT · Kubernetes Deployment
- Argo CD Automated Sync · Docker Build Cache